

# Coding Assignment 2

Due: Nov. 18, 23:59 PT

## Overview

This assignment includes 3 parts, covering Lesson 12 - Lesson 22. Unless specified, you can assume the inputs are always correct, so no input validation is needed. Please follow the specifications (in red) and make sure there is no compiler error in your code.

### You will receive a zero if...

- Your code is not runnable (wrong class/method name, wrong parameter & return type, compiler errors, etc.)
- Your code is from the Internet, AI, or other student.

## Setup

1. Either use the same project created for assignment 1 or create a new empty project.
2. Create a package called “**assignment2**”
3. Create 3 classes under the assignment1 package: “**MyRecursion**”, “**MyFile**”, and “**MyCollections**”
4. (Optional) Create a Main class with `psvm` to test your code (don't need to submit)

## Part 1. Recursion (2 pts)

In “**MyRecursion**”, implement a **public static** method called “**indexOf**” that **takes two Strings** as parameters and **returns an int** for the starting position of the first String that matches the second String. Return -1 if the second String is not a substring of the first String.

### Additional requirements

- Use recursion (Hint: implement a recursive helper method to keep track of how far the match is from the beginning of the text)
- Do NOT use the `indexOf` method in the Java library

### Examples

- Example 1: `int index = indexOf("Mississippi", "sip");`  
Since sip starts after the 6<sup>th</sup> letter of Mississippi, after executing the code, `index` should be 6.
- Example 2: `int index = indexOf("panda", "pan");`  
Since panda starts with pan, after executing the code, `index` should be 0.
- Example 3: `int index = indexOf("Hello world", "wow");`  
Since wow is not a substring of Hello world, after executing the code, `index` should be -1.

## Part 2. Read & Write a File (3 pts)

In “MyFile”, implement a **public static** method called “writePrimes” that takes two Strings for two filenames as parameters. The first file contains lines of words or integers ( $\geq 2$ ), separated by commas. Read the first file, and write any prime numbers to the second file, each on a new line.

### Additional requirements

- Use try-with-resource.
- Use two Scanner objects: one scans the file line by line, the other scans each line word by word.
- Has a separate method to check whether a “word” is a prime. The code should include minimum number of iterations needed to return a result.

### Example

Suppose the filename of the first file is “file1”, and here’s its content:

```
panda,CS47,2,hello,5,11WOW
yes,no,5,18,3
```

Then after executing `writePrimes("file1", "file2");` a file called “file2” will be created and here will be its content:

```
2
5
5
3
```

## Part 3. Use Collection Framework (5 pts)

This part includes 2 subparts (2.5 pts each). Your job also includes choosing an appropriate data structure and method(s) to use, according to lessons 20 - 22.

### Part 3.1: Downsize an employee collection

In “MyCollections”, implement a **public static void** method called “downsize” to remove every  $n^{\text{th}}$  element of a collection of employee names. The parameter is `Collection<String>` and an `int`, where `Collection` should be a specific type of data structure that is efficient to remove/add elements in the middle but not to access elements arbitrarily.

### *Additional Requirements*

Use a list iterator to go over the collection and remove elements. (Hint: you can have an `int` to keep track the original indexes)

### *Example*

Suppose `employees` is a collection of `String`, and it contains:

```
["Tom", "Bob", "Cathy", "Alice", "Trudy", "Harry", "Denny"]
```

Then calling `downsize(employees, 3)` will remove every 3rd element and `employees` becomes:

```
["Tom", "Bob", "Alice", "Trudy", "Denny"]
```

### Part 3.2: Check if the parentheses are balanced

In “**MyCollections**”, implement a **public static void** method called “**isBalanced**” to check whether the parameter **String** has balanced parentheses (i.e., **returns a boolean**). For simplicity, suppose we only have ( ) and [ ]s, not { }s. You don't need to evaluate if the expression is valid.

#### *Additional Requirements*

- Use the approach and appropriate data structure mentioned in the lecture
- Only use the methods that are in the required data structure class

#### *Examples*

(arguments are in green for easy-reading; can be other symbols between parentheses)

- `isBalanced("println(list.get(0))")` should return true
- `isBalanced("[ ] (3*5)^2")` should return true
- `isBalanced("[ (2+3)*5)^2")` should return false (wrong closing parenthesis)
- `isBalanced("println(list.get(0)")` should return false (missing closing parenthesis)
- `isBalanced("[ (3*5) ] ^2")` should return false (missing opening parenthesis);

### **Submission & Grading**

Submit `MyRecursion.java`, `MyFile.java`, and `MyCollections.java` to the corresponding Canvas assignment (Drag the files into the submission box and click “Upload”).

#### Auto-grading

You can submit as many times as you want to pass the auto-graded tests. No points back for any errors that are caught by the auto-grader.

#### Manually Grading

After the assignment is due, the grader will manually check your code. Since this is the second assignment, the requirement will be stricter. Here are some additional coding-style requirements other than the requirements for each part:

- No redundant/unnecessary code or unexpected/additional printouts.
- Your code should be formatted properly (i.e., have correct indentations and spaces). Use your IDE's reformat functionality if needed. For example, in IntelliJ, the shortcut is Ctrl + Alt + L on Windows and command + option + L on macOS.
- The required classes should not include `psvm`. If you'd like to run your methods, create a separate class that contains `psvm`.

For each part, you will get up to 1.5 pts (3 pts for part 3) deducted if your code doesn't satisfy any additional requirements. That is, your final score on the assignment may be lower than the auto-grading result but will not be higher.